

# Logical Replication In POSTGRESQL

Mini Project Report

By SAIDARSHAN MANJUNATH HEGDE

(24010303010)

Under the supervision of

**Dr. SRINATH M S**



SRI SATHYA SAI INSTITUTE OF HIGHER LEARNING

(Deemed to be University)

Department of Mathematics and Computer Science

PrasanthiNilayam Campus

April 2025

# Contents:

1.Introduction

2.Objective

3.PostgreSQL Setup

4.Logical Replication (DML)

5.Logical Replication (DDL)

6.Future Work

# 1. Introduction

PostgreSQL is a powerful, open-source object-relational database system known for its robustness, extensibility, and standards compliance. With over 30 years of active development, PostgreSQL has earned a strong reputation for reliability, performance, and advanced features. As an open-source project, it provides developers and database administrators with a high degree of flexibility and customization. Among its many features, one of the most critical for high availability and data distribution is replication. Logical replication, in particular, enables fine-grained control over data replication, making it ideal for modern data-driven applications.

Unlike physical replication, which copies the exact binary format of the database cluster, logical replication works at a higher level—sending individual changes such as row-level inserts, updates, and deletes. This allows replication between different major versions of PostgreSQL and provides selective replication capabilities (e.g., replicating specific tables or columns). PostgreSQL's open-source nature has also fostered a rich ecosystem of extensions, such as `pglogical`, which further enhances logical replication by supporting DDL propagation and more complex replication topologies.

## 2. Objective

The objective of this mini-project is to explore and implement logical replication in PostgreSQL, with a specific focus on replicating both DML (Data Manipulation Language) operations and DDL (Data Definition Language) changes. While native PostgreSQL logical replication supports only DML by default, this project utilizes the `pglogical` extension to enable DDL replication as well. This ensures that schema changes such as table creation or modification are automatically reflected on subscriber nodes, streamlining database synchronization and minimizing manual interventions.

### 3. PostgreSQL Setup

To demonstrate logical replication, PostgreSQL was first built and installed from source to ensure full control over features and extensions like `pglogical`. Two PostgreSQL instances were configured:

- **Publisher:** Runs on port 5432 with its own data directory
- **Subscriber:** Runs on port 5433 with a separate data directory

#### Installing PostgreSQL from Source

Download and extract the PostgreSQL source code:

```
wget https://ftp.postgresql.org/pub/source/v16.2/postgresql-16.2.tar.gz  
tar -xzf postgresql-16.2.tar.gz  
cd postgresql-16.2
```

The installation process began by downloading the official PostgreSQL source code from the PostgreSQL website. The archive was extracted and the source directory was entered. Configuration was done using the `configure` script with a custom installation path pointing to the user's home directory.

Once configured, PostgreSQL was compiled using the `make` command with parallel jobs enabled to speed up the process. After a successful build, it was installed using `make install` command.

#### Initializing PostgreSQL Instances

Two separate data directories were created—one for the publisher instance and another for the subscriber. Both were initialized using the `initdb` command. The configuration files located in each data directory (`postgresql.conf` and `pg_hba.conf`) were edited to enable logical replication. Important parameters such as `wal_level` were set to `logical`, and `shared_preload_libraries` was set to include `pglogical`.

The `postgresql.conf` file was also modified to define unique port numbers for each instance (5432 for the publisher and 5433 for the subscriber), and to set values for `max_replication_slots` and `max_wal_senders`, which are essential for replication support.

Access permissions were configured in the `pg_hba.conf` file to allow local trust authentication for replication and normal connections.

## Starting the PostgreSQL Instances

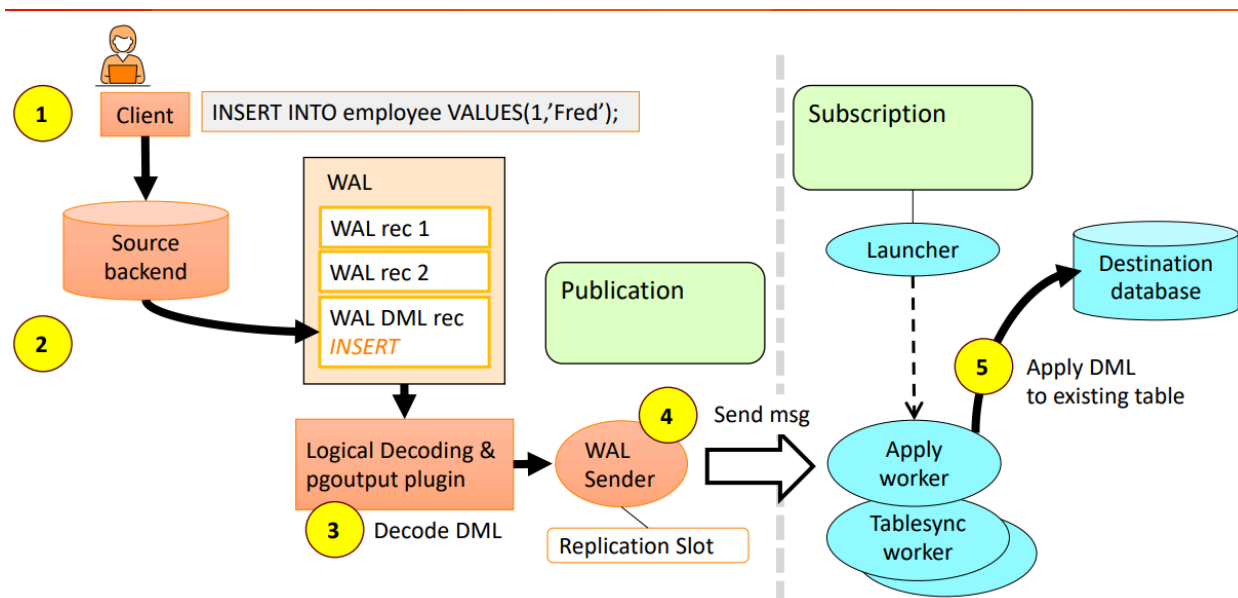
After configuration, both PostgreSQL instances were started using the `pg_ctl` command, each with its respective data directory and log file. This created two independently running servers on the same machine.

## Installing and Configuring pglogical

To support advanced logical replication features, the `pglogical` extension was cloned from its official repository. It was then compiled and installed. Depending on the system configuration, certain binaries such as `pglogical_create_subscriber` and shared libraries needed to be manually copied into the appropriate PostgreSQL lib and extension directories.

With both instances running and `pglogical` installed, the environment was ready for testing logical replication, including both DML and DDL operations.

# 4. Logical Replication (DML)



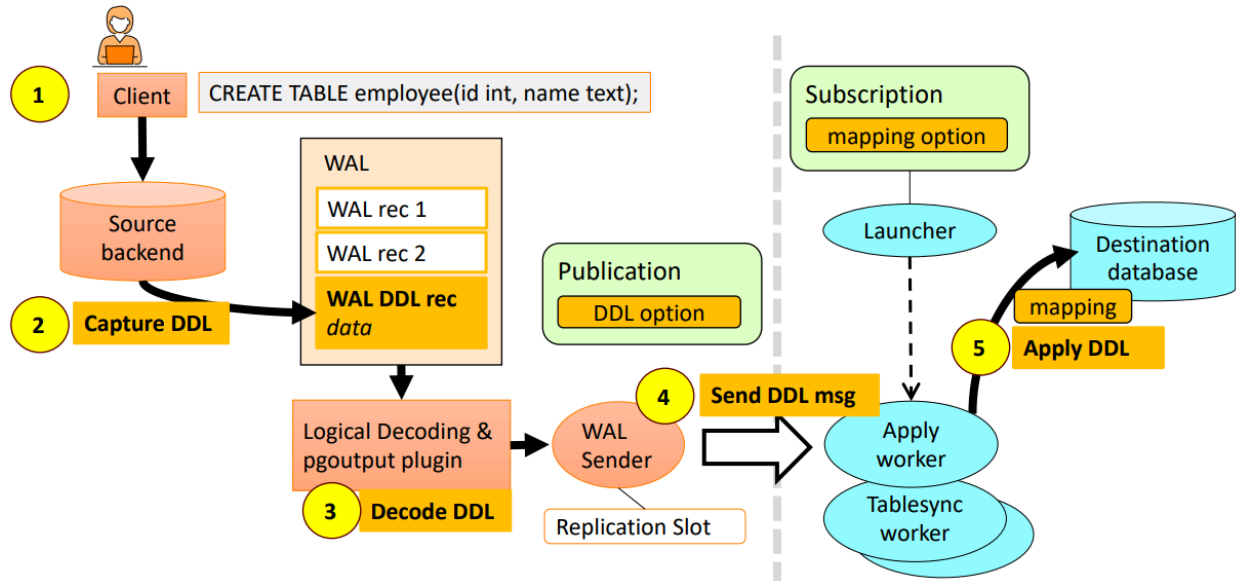
Logical replication for DML (Data Manipulation Language) operations in PostgreSQL involves replicating changes such as INSERT, UPDATE, DELETE, and TRUNCATE from a **Publisher** to a **Subscriber**.

The typical flow of DML replication is:

1. **Client** – The client issues a DML query such as:  
INSERT INTO employee VALUES(1, 'Fred');
2. **Source Backend** – The DML operation is executed on the publisher database and is written to the Write-Ahead Log (WAL).
3. **WAL (Write-Ahead Log)** – WAL stores all changes, including the DML record.
4. **Logical Decoding & Output Plugin (e.g., pgoutput)** – PostgreSQL decodes the logical changes using the output plugin and prepares them for replication.
5. **WAL Sender Process** – The decoded message is sent to the subscriber using the replication protocol.
6. **Subscription & Launcher** – On the subscriber side, the launcher process starts the apply and tablesync workers.
7. **Apply Worker** – The changes are applied to the destination table in the subscriber database.

This model ensures real-time data synchronization while allowing for selective replication (specific tables, columns, etc.).

## 5. Logical Replication (DDL)



By default, PostgreSQL logical replication does not support DDL replication. However, using the pglogical extension, this limitation can be overcome to support schema changes such as CREATE, ALTER, or DROP.

The flow of DDL replication using pglogical:

1. **Client** – Executes a DDL command like:  
`CREATE TABLE employee(id INT, name TEXT);`
2. **Source Backend** – The backend captures the DDL operation and records it in WAL.
3. **WAL** – Contains DDL records alongside regular WAL entries.
4. **Logical Decoding & pgoutput Plugin** – Decodes the DDL operation.
5. **WAL Sender** – Sends the DDL message across the replication slot.

6. **Subscription with Mapping Option** – The subscriber maps the received DDL messages using the provided mapping configuration.
7. **Apply Worker** – Applies the DDL command on the subscriber, ensuring schema consistency.

With this extended capability, database administrators can ensure that not only the data but also the structure of the database remains synchronized across nodes.

Each component in both diagrams plays a specific role:

- **WAL** ensures durability and consistency.
- **Logical Decoding** transforms WAL into a readable format.
- **WAL Sender** communicates changes.
- **Subscription** and **Apply Workers** maintain replica state in sync.

## 6.Future Work

In future iterations, the logical replication setup can be improved by automating the entire process using shell or Python scripts to make it production-ready. Monitoring tools such as `pg_stat_replication` or third-party dashboards can be integrated to track replication lag and detect conflicts. Bi-directional replication setups may be explored for high availability use cases. The solution can also be extended to support replication across multiple tables and schemas. Additionally, contributing to PostgreSQL's core development to enhance built-in DDL replication support would be a meaningful long-term goal.